NAME : SAFIYA B

REG NO: 192411043

COMPARATIVE ANA

Q33. Kruskal vs Borůvka Algorithm – MST Comparison

Aim

To construct the **Minimum Spanning Tree (MST)** of a weighted graph using **Kruskal's and Borůvka's algorithms**, compare their edge-selection processes, analyze efficiency, computational complexity, and scalability.

Given Graph

Consider the following weighted graph:

Edge	Weight
0–1	4
0–2	4
1–2	2
1–3	5
2–3	8
2–4	10
3–4	2
3–5	6
4–5	3

Number of vertices: **6**

1. Kruskal's Algorithm

Kruskal's algorithm sorts all edges by increasing weight and repeatedly selects the smallest edge that does not form a cycle.

Sorted edges

1–2 → 2

3–4 → 2

4–5 → 3

0–1 → 4

0–2 → 4

1–3 → 5

3–5 → 6

2–3 → 8

2–4 → 10

Edge Selection

Step	Edge	Weight	Action
1	1–2	2	Selected
2	3–4	2	Selected
3	4–5	3	Selected
4	0–1	4	Selected
5	0–2	4	Rejected – cycle

We already have **V − 1 = 5** edges.

MST

1–2 = 2

3–4 = 2

4–5 = 3

0–1 = 4

1–3 = 5

Total MST Weight

$$2 + 2 + 3 + 4 + 5 = \boxed{16}$$

2. Borůvka's Algorithm

Borůvka's algorithm initially considers every vertex as a separate component. In every phase, each component selects its **cheapest outgoing edge**.

Phase 1

The components choose their cheapest edges:

Component 0 → 0–1 (4)

Component 1 → 1–2 (2)

Component 2 → 1–2 (2)

Component 3 → 3–4 (2)

Component 4 → 3–4 (2)

Component 5 → 4–5 (3)

After merging, the number of components decreases.

Phase 2

The remaining components choose their cheapest connecting edge:

Component {0,1,2} → 1–3 (5)

Component {3,4,5} → 1–3 (5)

Now all vertices belong to one component.

MST

1–2 = 2

3–4 = 2

4–5 = 3

0–1 = 4

1–3 = 5

Total:

$$\boxed{16}$$

Thus, both algorithms produce an MST with the same minimum total weight.

3. Comparative Analysis

Feature	**Kruskal**	**Borůvka**
Selection method	Globally smallest edge	Cheapest edge of each component
Sorting	Requires edge sorting	No complete sorting required
Cycle detection	Union-Find	Union-Find
Time complexity	O(E log E)	O(E log V)
Parallel processing	Less suitable	Highly suitable
Implementation	Simple	Slightly complex
Sparse graphs	Very effective	Effective
Dense/large graphs	Can become expensive	Good scalability
Distributed systems	Moderate	Very suitable

4. Efficiency for Different Graph Types

Sparse Graph

A sparse graph contains relatively few edges. **Kruskal** performs well because sorting a small number of edges is inexpensive.

Dense Graph

A dense graph contains many edges. Sorting all edges in Kruskal can become costly. Borůvka can be advantageous because components independently search for their cheapest outgoing edges.

Large Graph

For very large graphs, Borůvka is attractive because several components can process their cheapest edges simultaneously.

Parallel Graph

Borůvka is particularly suitable for parallel systems because the cheapest edge of different components can be determined independently.

5. Computational Complexity

Let:

- **V** = number of vertices
- **E** = number of edges

Kruskal

Sorting edges requires:

$$O(E \log E)$$

Therefore:

$$\boxed{O(E \log E)}$$

Borůvka

Each phase takes approximately:

$$O(E)$$

The number of components decreases significantly in each phase, resulting in approximately:

$$\boxed{O(E \log V)}$$

6. Scalability

Kruskal:

- Easy to implement.
- Good for small and medium graphs.
- Efficient for sparse graphs.
- Edge sorting can become a bottleneck for extremely large graphs.

Borůvka:

- Works well for large graphs.
- Naturally supports parallel processing.
- Suitable for distributed graph-processing systems.
- Can handle large-scale MST construction efficiently.

7. Insights

1. Both algorithms produce the **minimum spanning tree**.
2. Kruskal selects edges based on **global minimum weight**.
3. Borůvka selects the **cheapest outgoing edge for every component**.
4. Kruskal is easier to implement and understand.
5. Borůvka provides better opportunities for **parallel processing**.
6. Kruskal is particularly effective for sparse graphs.
7. Borůvka is a strong choice for large-scale and distributed graphs.
8. Both algorithms contain exactly **V − 1 edges** in the final MST.
9. For the given graph, both algorithms produce an MST of total weight **16**.

Conclusion

Kruskal and Borůvka are efficient MST algorithms but use different strategies. **Kruskal follows a global edge-based approach**, while **Borůvka follows a component-based approach**. Kruskal is simpler and works very well for sparse graphs, whereas Borůvka is more suitable for large, parallel, and distributed graph-processing applications.